CS4375 Fall 2025 Homework Report
<Gabriel Peña > Submitted on <12/08/2025>
<gpena6@miner.utep.edu >

Semaphores for xv6

Task 3. Implementation of sem_init(), sem_wait(), sem_post(), and sem_destroy().
The function sem_wait() takes control of the semaphore's lock and looks at the counter. When the count is 0, the process that is calling goes to sleep on the semaphore by using sleep(s, &s->lock). This action releases the lock and puts the process to sleep at the same time. When it wakes up, it checks the counter again, lowers it by one, and then unlocks it.
The function sem_post() locks the semaphore, increases its count, and then wakes up any processes that are waiting on this semaphore. "This lets the processes that are waiting in sem_wait() continue."
Some difficulties that I came across just making changes from one file to another, and if one problem occur then fix another after another

Task 4. Readers–Writers with xv6 Semaphores.
Explain your implementation of reader() and writer function. Show your outputs in the report, you can implement print statements in reader/write function to show intermedidate values. You would also need to submit you completed rwtest-sem. Function.

Summary:
In this lab, I discovered how xv6 handles system calls from start to finish. This includes creating prototypes for users, making syscall stubs, and linking them to functions in the kernel. I also discovered how to create kernel data structures, such as the semaphore table, and how to safely handle shared information using spinlocks. By using sem_wait() and sem_post(), I learned how xv6 employs sleep() and wakeup() to pause and resume processes. This experience helped me better understand how the kernel manages synchronization and handles multiple tasks at the same time. Sure! Please provide the text you'd like me to paraphrase.